

Ontologia e Aggregazione dei Knowledge Graph

Zazzarini Micol

February 11, 2026

1 Ontologia

Viene costruito un **Knowledge Graph (KG)** a partire da due tipi di dati:

1. **Entità deterministiche:** Queste entità sono **inserite nel testo** secondo pattern definiti, e rappresentano informazioni **deterministiche** sull'esecuzione. Non derivano dal reasoning dell'agente e non devono essere inferite automaticamente:
 - `@run(id=<RUN_ID>)`
 - `@step(id="<STEP_ID>", index=<STEP_INDEX>)`
 - `@agent(id="<AGENT_ID>", model="<MODEL>")`
 - `@user(id=<USER_ID>)`
 - `@task(id=<TASK_ID>, description=<DESC>)`
 - `@state(...)`
 - `@observation(tool=<TOOL>, output=<OUTPUT>)`
 - `@termination`
2. **Reasoning:** testo delimitato da `@start_reasoning(step_id=<id>)` e `@end_reasoning(step_id=<id>)`.

Le entità non vengono **mai inferite o ricostruite al di fuori dei pattern o dei blocchi di reasoning**.

1.1 Entità

- **trace:Run:** Estratta dal pattern `@trace.run(id=<RUN_ID>)`. Contiene tutti gli Steps e le entità relative a un'esecuzione.
- **trace:Step:** Estratta dal pattern `@trace.step(id="<STEP_ID>", index=<STEP_INDEX>)`. Rappresenta un'unità atomica di reasoning.
- **trace:Agent:** Estratta dal pattern `@trace.agent(id="<AGENT_ID>", model="<MODEL>")`. Rappresenta l'agente.
- **trace:User:** Estratta dal pattern `@trace.user(id=<USER_ID>)`. Rappresenta l'utente che richiede il Task.
- **mind:Statement:** Estratta dai blocchi di reasoning. Rappresenta le affermazioni auditable che non sono Goal o Intention.

- **mind:Goal:** Estratta dai blocchi di reasoning. Rappresenta un determinato obiettivo desiderato.
- **mind:Intention:** Estratta dai blocchi di reasoning. Rappresenta l'intenzione di eseguire una certa azione.
- **mind:TerminationSignal:** Estratta dal pattern `@termination`. Indica che una Run è completata.
- **act:Tool:** Estratta dal reasoning; deve corrispondere ai tool canonici (calculator, ecc...).
- **act:ToolCall:** Estratta dal reasoning; rappresenta l'invocazione concreta del Tool.
- **obs:Observation:** Estratta dal pattern `@observation(tool=<TOOL>, output=<OUTPUT>)`. Rappresenta il risultato ottenuto da un tool.
- **state:State:** Estratta dal pattern `@state(id=<id>, calculator_ready=<calculator_ready>, ...)`. Rappresenta lo stato corrente dell'agente con eventuali variabili.
- **state:Calculator_ready:** Estratta dal pattern `@state(id=<id>, calculator_ready=<calculator_ready>, ...)`. Rappresenta una delle eventuali variabili dello stato dell'agente.
- **task:Task:** Estratta dal pattern `@task(id=<TASK_ID>, description=<DESC>)`. E' la richiesta dell'utente, che rappresenta il task da eseguire/risolvere.

1.2 Relazioni

Ogni relazione è definita con:

- **Dominio:** classe del soggetto
- **Range:** classe dell'oggetto
- **Direzione:** soggetto $\rightarrow$ oggetto
- **Quando creare:** regole specifiche, obbligatorie o condizionali

Le relazioni definite sono:

- **trace:hasStep:** Ogni Run viene associata agli Steps che la compongono.
- **trace:performedBy** (Run $\rightarrow$ Agent): Lega l'agente alla propria esecuzione.
- **trace:nextStep** (Step $\rightarrow$ Step): Serve per aggregare i grafi generati da ogni reasoning/step, e lo fa ordinando gli steps per indice/ordine sequenziale.
- **mind:assertedIn** (Statement $\rightarrow$ Step): Indica in quale step/reasoning è stato estratto un determinato Statement.
- **mind:expressedIn** (Intention $\rightarrow$ Step): Indica in quale reasoning è stata estratta una determinata Intention
- **mind:motivates** (Goal $\rightarrow$ Intention) : Indica che l'intenzione di eseguire una determinata azione viene espressa dall'agente per raggiungere un determinato Goal.
- **mind:terminates** (TerminationSignal $\rightarrow$ Run): Termine della Run corrente.

- `mind:goalInStep` ($\text{Goal} \rightarrow \text{Step}$): Indica in quale reasoning è stato estratto un determinato Goal.
- `act:usesTool` ($\text{ToolCall} \rightarrow \text{Tool}$): Indica quale specifico Tool (tra quelli predefiniti) è stato chiamato attraverso una ToolCall.
- `act:usedInStep` ($\text{ToolCall} \rightarrow \text{Step}$): Indica in quale step l'agente ha effettuato una ToolCall.
- `obs:observedIn` ($\text{Observation} \rightarrow \text{Step}$): Indica in quale step l'agente ha ricevuto risposta da un Tool chiamato precedentemente attraverso una ToolCall.
- `obs:results` ($\text{ToolCall} \rightarrow \text{Observation}$): Lega una determinata ToolCall al suo risultato.
- `state:ownedBy` ($\text{State} \rightarrow \text{Agent}$): lega l'agente al proprio stato.
- `state:hasVariable` ($\text{State} \rightarrow \text{StateVariable}$): Lega un determinato stato dell'agente ai valori delle sue variabili.
- `state:activeIn` ($\text{State} \rightarrow \text{Step}$): Indica che in un determinato step della propria esecuzione l'agente possiede un determinato stato.
- `task:requestedBy` ($\text{Task} \rightarrow \text{User}$): Lega l'utente a ciò che l'utente stesso ha richiesto all'agente.
- `task:handledByRun` ($\text{Task} \rightarrow \text{Run}$): Indica che il task specifico (o quale task specifico) richiesto dall'utente viene eseguito nella corrente Run.

Tutte le relazioni rispettano dominio, range e direzione esattamente come definito nei prompt.

2 Aggregazione dei grafi

La funzione `aggregate(self, graphs: list[Graph])` (in `kg_gen.py`) unisce più grafi in un singolo grafo aggregato:

1. Ogni entità estratta durante il processo di reasoning riceve un **ID univoco**, ottenuto aggiungendo il suffisso `_G{i}` (dove *i* identifica lo specifico step o reasoning di provenienza). Questo meccanismo è necessario per evitare che entità semanticamente simili o con lo stesso nome (ad esempio `mind:Statement`, `mind:Goal`, ecc.) vengano automaticamente **aggregate in un unico nodo del grafo**.

Durante l'estrazione da più step di reasoning, infatti, possono comparire entità dello stesso tipo che rappresentano però **istanze distinte**, ciascuna legata a uno specifico contesto o momento del processo inferenziale.

Senza l'aggiunta di un identificatore univoco:

- tutte le entità con lo stesso nome verrebbero fuse;
- si perderebbe l'informazione sullo specifico step da cui provengono;
- verrebbe compromessa la corretta rappresentazione della struttura del reasoning.

Con l'aggiunta del suffisso `_G{i}`, invece:

- ogni entità mantiene la propria identità;
- si preserva il legame con lo specifico reasoning di origine;

- il grafo risultante rappresenta fedelmente la struttura sequenziale o ramificata del processo inferenziale.

In questo modo, anche entità appartenenti allo stesso tipo ontologico rimangono **separate a livello di istanza**, evitando collisioni semantiche e garantendo una modellazione più accurata del processo di reasoning.

2. Tutte le entità, relazioni e metadati vengono copiate con i nuovi ID.
3. Gli Steps vengono ordinati numericamente in base all'indice.
4. Si aggiungono relazioni strutturali:
 - `trace:nextStep` tra Steps consecutivi
 - `trace:hasStep` dal Run a tutti gli Steps
 - `trace:endsWith` dal Run ai TerminationSignal
5. Le entità duplicate in Steps diversi restano separate, e legate allo step che rappresenta il reasoning da cui sono state estratte.